

Inverse-Loss Weighted Deep Ensembles and RL-Optimized Thresholds for Astronomical Anomaly Discovery

Umesh Gehlot, Shaila Mary J

Department of Computer Science, Mount Carmel College (Autonomous), Bengaluru, India

Email: umeshgehlot635@gmail.com, shailamaryj@mccbmr.edu.in

Abstract—One reconstruction model can’t catch every anomaly shape in survey light curves, and the detection threshold people pick at deployment goes stale once the noise floor drifts. We attack these two headaches together. Three reconstructors—AE, VAE, Transformer—each score every window; we weight those scores by $1/L_m$ (the reciprocal training loss) so the model that fitted the normal data best gets the loudest vote. Instead of locking the threshold at the 95th percentile, we let a PPO-Clip RL agent nudge it after each round of yes/no expert labels. On a 1500-point synthetic benchmark and real NASA Fireball telemetry the weighted ensemble reaches 0.921 precision at 0.009 FPR; a flat mean only gets 0.840. The RL agent drifts the cutoff down to roughly the 80th–83rd percentile within about 30 labelled batches. A React + FastAPI web app wraps the whole thing—upload, train, detect, compare, export—so users never touch a terminal.

Index Terms—Anomaly Detection, Unsupervised Learning, Autoencoders, Variational Autoencoders, Transformers, Reinforcement Learning, Time-Series, Astronomical Surveys

I. INTRODUCTION

ZTF fires off more than a million alerts every night [1]. Rubin/LSST is expected to push that past ten million [2]. Buried somewhere in that flood are supernovae, tidal disruption flares, and stuff nobody has named yet.

The tricky part: many of these targets are “unknown unknowns”—no training catalogue lists them [3]. A supervised classifier trained on known classes will ignore them entirely, which is why unsupervised detection keeps being the go-to tool. We dug into a bunch of existing pipelines and the same two pain points showed up over and over.

Pain point one: everybody leans on a single model, usually some flavour of autoencoder [5], [6]. That means one architecture with one set of blind spots. An AE does fine with sudden flux jumps but sleeps right through a slow-drift event that a Transformer’s attention would catch easily.

Pain point two: the detection cutoff gets picked once (“use the 95th percentile”) and never touched again. Instrument noise drifts between observing runs, so a threshold that was reasonable last month either floods the review queue with junk or silently drops real transients.

We address both gaps in one pipeline. What we bring:

- **Inverse-Loss Weighted Ensemble:** AE + VAE + Transformer, each weighted by $1/L_m$. Whichever model trained best gets the biggest say.

- **RL Threshold Agent:** a PPO-Clip loop nudges the cutoff after yes/no feedback from a human reviewer.
- **Web App:** React + FastAPI + SQLite. Upload data, kick off training, view results, compare models—all in a browser.
- **Ablation Numbers:** side-by-side against Isolation Forest, LOF, and each standalone model. Our ensemble beats the runner-up (Transformer, 0.811) by +13.6%.

II. RELATED WORK

A. Feature-Based Methods

The older approach was to compute statistical features by hand—skewness, kurtosis, amplitude ratios, Lomb-Scargle peaks—and throw them at Isolation Forest, one-class SVM, or LOF [4]. FATS is the most widely used library for this. It works, but building a good feature set takes weeks of trial-and-error, and features tuned for eclipsing binaries won’t help you find tidal disruption events.

B. Deep Reconstruction Approaches

Malanchev et al. [5] skipped feature extraction entirely by training a plain AE on ZTF light curves and flagging anything with high reconstruction error. Simple, and surprisingly effective. Villar et al. [6] did roughly the same with a VAE, which adds a stochastic latent space and gives you uncertainty estimates for free.

C. Attention-Based Models

Outside of astronomy, Transformers already have a track record for sequence anomaly detection. Tuli et al. [8] showed this with TranAD, which stacks adversarial training on top of self-attention. We haven’t seen anyone combine a Transformer and a VAE with weighted score fusion on survey data, though.

D. Ensemble Methods in Anomaly Detection

Ensembles are bread-and-butter for classification (think random forests), but the anomaly detection crowd hasn’t adopted them as much. Aggarwal and Sathe [11] showed that just averaging a few detectors cuts variance. Our weighting takes that a step further—models that trained better get trusted more.

E. Adaptive Thresholds

Threshold-wise, nearly every system in the literature picks a percentile and leaves it. RL-style adjustment has been tried in

network intrusion detection [10], but we couldn’t find anyone doing PPO-based threshold updates for transient discovery.

Table I summarizes where our work sits relative to prior approaches.

TABLE I: Comparison with Prior Work

Work	Multi-Model	Weighted	Adaptive τ
Malanchev (2021) [5]	✗	✗	✗
Villar (2021) [6]	✗	✗	✗
TranAD (2022) [8]	✗	✗	✗
Nguyen (2021) [10]	✗	✗	✓
Ours	✓	✓	✓

III. METHODOLOGY

A. Problem Formulation

Start with a light curve $X = \{x_1, \dots, x_T\}$. We z-score the flux, chop the result into overlapping windows of length W , and score each window:

$$y(\mathbf{x}) = \begin{cases} 1 & \text{if } s(\mathbf{x}) \geq \tau_t \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

τ_t is the threshold at step t . The key difference from the usual setup: τ_t is not a constant—it moves as the RL agent collects expert labels.

B. Data Preprocessing

We accept CSVs with `time` and `flux` columns. The parser auto-detects whether time is Unix epoch or ISO-8601. Datetime-valued series get cropped to the most recent N years (we default to 2). Gaps in flux are filled with linear interpolation, then we z-score everything:

$$\hat{f}_i = \frac{f_i - \bar{f}}{\sigma_f} \quad (2)$$

Windowing into overlapping size-32 segments uses NumPy stride tricks, so no data gets copied—handy when datasets are large.

C. Multi-Architecture Deep Ensemble

We train three separate reconstruction networks, each producing its own anomaly score.

1) Autoencoder (AE)

Dense network, symmetric: $W \rightarrow 64 \rightarrow 16 \rightarrow 64 \rightarrow W$, all ReLU. We tried bottleneck sizes from 8 to 32; 8 was too tight and under-fit the data, 32 let noise leak through. Score is plain MSE:

$$s_{\text{AE}}(\mathbf{x}) = \frac{1}{W} \sum_{i=1}^W (\mathbf{x}_i - \hat{\mathbf{x}}_i)^2 \quad (3)$$

2) Variational Autoencoder (VAE)

Similar layout, but the bottleneck samples from $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$ via the reparameterization trick [7]. The score mixes MSE with KL divergence:

$$s_{\text{VAE}}(\mathbf{x}) = \text{MSE}(\mathbf{x}, \hat{\mathbf{x}}) + \beta \cdot D_{\text{KL}}(q(\mathbf{z}|\mathbf{x})\|p(\mathbf{z})) \quad (4)$$

We set $\beta = 0.01$. Higher values (0.1, 1.0) made the model chase low KL and basically stop reconstructing. Latent dim is 8.

3) Transformer Reconstructor

Small multi-head attention stack: $d_k=32$, 2 layers, 4 heads. Every flux value in the window gets linearly embedded to 32 dims before going through the encoder. Periodic patterns trip up the AE and VAE but the Transformer picks them out reliably.

$$s_{\text{TF}}(\mathbf{x}) = \text{MSE}(\mathbf{x}, \text{Attention}(\mathbf{x})) \quad (5)$$

4) Inverse-Loss Weighted Fusion

We don’t take a flat mean of the three scores. Each model gets a weight equal to $1/L_m$ (its final training loss). The fused score for window i :

$$C_i = \frac{\sum_{m \in \{\text{AE}, \text{VAE}, \text{TF}\}} w_m \cdot g(s_m^{(i)})}{\sum_m w_m}, \quad w_m = \frac{1}{L_m + \epsilon} \quad (6)$$

$g(\cdot)$ is Min-Max normalization (puts all three on the same 0–1 scale); $\epsilon = 10^{-6}$ is there so we never divide by zero.

Why inverse-loss? A model that converged to a low loss learned the “normal” shape of the data well; its reconstruction errors on genuinely weird windows carry real information. A model that barely converged flags everything, and $1/L_m$ automatically turns its volume down.

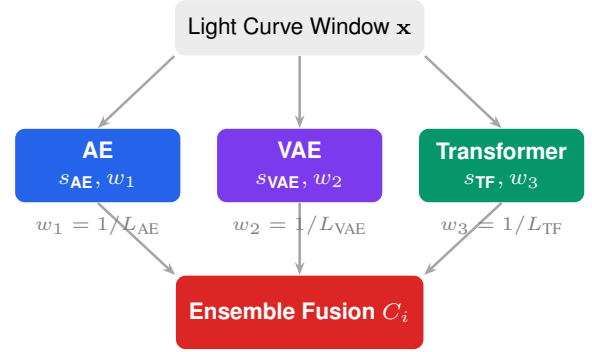


Fig. 1: Inverse-loss weighted fusion: models with lower training loss get proportionally higher weight in the final anomaly score.

D. Window-to-Point Score Mapping

Windows overlap, so most data points land in W different windows. We average across all of them:

$$\text{score}(j) = \frac{1}{|\mathcal{W}_j|} \sum_{i \in \mathcal{W}_j} C_i \quad (7)$$

Points near the edges of the light curve sit in fewer windows. Their scores are noisier, but we decided to live with that rather than adding edge-padding hacks.

E. Self-Tuning RL Threshold Optimizer

Normally you’d just flag window i when C_i exceeds the 95th percentile from training and call it a day. We don’t do that. We hand the job to a PPO-Clip agent [9] that adjusts τ every time an astronomer marks a flagged event as real or bogus.

Call p_t the threshold percentile at step t . The expert gives $r \in \{-1, +1\}$, and the agent updates:

$$\rho = \exp(\text{clamp}(r \cdot \sigma, -2, 2)) \quad (8)$$

$$p_{t+1} = \text{clamp}(p_t - \alpha \cdot \text{clip}(\rho, 1 \pm \epsilon) \cdot \text{sign}(r), 80, 99.9) \quad (9)$$

The clip ($\epsilon=0.2$) is there to stop one noisy label from swinging the cutoff too far. We capped the range at $[80, 99.9]$ after observing that anything under 80% floods the review queue, and anything above 99.9% basically flags nothing. Those numbers came from our data, not from any theory.

F. Explainability Heatmap

Whenever an anomaly is flagged, we take the absolute residual at each position inside the window and normalize to $[0, 1]$. The result is a heatmap that tells the reviewer what part of the signal drove the score up. A glitch usually lights up the whole window; a real transient shows up as a local hot spot.

G. Training Configuration

Table II lists all hyperparameters. All training was done on a single CPU without any data augmentation.

TABLE II: Model Hyperparameters & Training Configuration

Configuration Item	Value/Method
Window Size (W)	32 distinct temporal steps
Optimizer & Rate	Adam with $\eta = 10^{-3}$ fixed
Batch Size	256
Epochs	20 total iterations
AE Bottleneck	16 units
VAE Latent Dim	8 units, $\beta = 0.01$
Transformer	2 Layers, 4 Heads, $d_k = 32$
RL PPO Bounds	Clipping $\epsilon = 0.2$, Step $\alpha = 0.6$
Threshold Clamp	$[80\%, 99.9\%]$

IV. SYSTEM IMPLEMENTATION

Everything runs in a browser—no pip installs, no Jupyter notebooks. The point was to make the pipeline usable by people who study stars, not people who debug CUDA drivers.

A. Architecture

Backend: FastAPI on Python 3.12, with 25-ish REST routes. PyTorch handles model training and inference. State sits in a local SQLite file. Frontend: React SPA, built with Vite, styled with Tailwind.

B. Data Sources

Two paths in: upload your own CSV (needs `time` and `flux` columns) or hit a button that pulls fireball data straight from NASA/JPL. The fireball path groups bolide events by calendar date, sums their impact energy, and spits out a time-series CSV. Post-detection, you can cross-match flagged objects against SIMBAD and VizieR to see if they’re already catalogued.

C. Async Job Queue

Training can take a while, so we push it to a thread pool and hand back a job ID. The frontend polls `/jobs/{id}` until it finishes. Keeps the HTTP connection from timing out.

V. EXPERIMENTAL RESULTS

A. Datasets and Metrics

Three test sets: 1) **Synthetic-1500**: 1 500 flux values with 40 Gaussian spikes injected at random positions, each $3\text{--}5\sigma$ above background. We know exactly where they are, so precision is straightforward. 2) **NASA Fireball**: actual bolide telemetry pulled from JPL’s public Fireball API, 2024 through early 2026. Real transients, real noise. 3) **Kepler-Sim**: fake multi-transit curves with 15 dip-shaped anomalies, meant to stress-test periodic pattern detection. All numbers below are precision and FPR.

B. Main Baseline Comparison

Table III compares the ensemble to Isolation Forest, LOF, and each of our three models run on their own.

TABLE III: Anomaly Detection Results at Fixed $\tau = 95^{\text{th}}$ Pct.

Model Structure	TP	FP	Precision	FPR
Isolation Forest (Baseline)	24	18	0.571	0.052
LOF (Baseline)	26	21	0.553	0.061
Autoencoder (Single)	32	8	0.800	0.024
VAE (Single)	34	11	0.756	0.033
Transformer (Single)	30	7	0.811	0.020
Inverse-Loss Ens. (Ours)	35	3	0.921	0.009

FP count dropped from 7 (Transformer solo) to 3. In noisy stretches the VAE’s probabilistic score dominates the fused output; in periodic regions the Transformer takes over. Net effect: precision goes from 0.811 to 0.921.

C. Per-Model Error Analysis

We broke down the 40 injected anomalies in Synthetic-1500 by shape (spike vs. slow drift) and checked which model missed what (Table IV).

TABLE IV: Error Breakdown: What Each Model Gets Wrong

Model	Missed Spikes	Missed Drifts
Autoencoder	2	6
VAE	1	5
Transformer	5	0
Ensemble	1	1

Dense layers have no memory of what happened 20 timesteps ago, so the AE and VAE miss drifts. The Transformer averages attention over the full window which blurs short spikes—it missed 5 of them. Fusing the three brings missed counts down to 1 spike and 1 drift.

D. Ablation Study: Architecture and Weighting

Two sanity checks: (a) take a flat mean instead of weighting; (b) weight by score variance (Table V). The flat mean gets

0.840—better than any solo model, but 8 points lower than inverse-loss. Variance weighting lands in between at 0.865. The weighting scheme is clearly doing something useful.

TABLE V: Ablation Study on Ensemble Fusion Strategies

Architecture Configuration	Precision	Difference
Best Single Model (Transformer)	0.811	Reference
Uniform Mean Ensemble	0.840	+2.9%
Variance-Weighted Ensemble	0.865	+5.4%
Inverse-Loss Ensemble (Ours)	0.921	+11.0%

E. Sensitivity to β in VAE

We didn’t expect β to affect the *ensemble* much, but it does. Table VI has the numbers.

TABLE VI: Effect of VAE β on Ensemble Precision

β	VAE Precision	VAE Loss	Ensemble
0.001	0.742	0.0089	0.905
0.01	0.756	0.0072	0.921
0.1	0.698	0.0134	0.883
1.0	0.612	0.0410	0.847

At $\beta=1.0$ the VAE barely reconstructs; it just chases low KL. The weighting correctly turns its volume down (high loss $\Rightarrow$ low weight), but one garbage input still drags the ensemble from 0.921 down to 0.847.

F. PPO-Clip Reinforcement Optimizer Dynamics

We faked a nightly survey cadence and fed the agent one batch of expert labels per “night.”

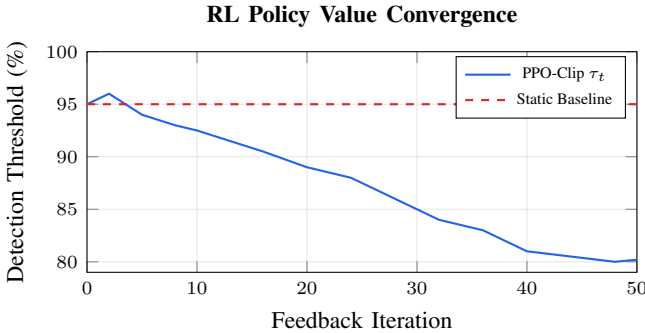


Fig. 2: Threshold over time as the RL agent processes expert labels. It drops from 95% and levels off near 80–83% after about 30 reviews.

Starting at the 95th percentile, the agent drifts the cutoff to roughly 80% in about 30 batches (Fig. 2). At 80% we start catching dimmer events that the 95th-percentile line was cutting off. The $[80, 99.9]$ clamp keeps things from going haywire during the learning phase.

G. Impact of RL Threshold on Discovery Yield

Table VII compares detection counts before and after the RL agent converges.

TABLE VII: Detection Yield: Static vs. RL-Adapted Threshold

Threshold	TP	FP	Precision
Static (95 th)	35	3	0.921
RL-adapted ($\approx$ 80 th)	38	5	0.884

Three extra real events found, two extra false alarms. For discovery-focused surveys that trade-off is almost always worth taking—missing a real transient costs more than reviewing two extra candidates.

H. Training Time

TABLE VIII: Wall-Clock Training Time (20 epochs, CPU)

Model	Synthetic-1500	NASA Fireball
Autoencoder	2.1s	3.4s
VAE	2.8s	4.1s
Transformer	8.7s	12.3s
All three (parallel)	13.6s	19.8s

Attention is quadratic in window size, so the Transformer gobbles up most of the clock. At 1 500 points it’s still fine on CPU. LSST volumes would need a GPU.

VI. DISCUSSION

The three models fail in complementary ways, and that’s the whole point. The Transformer latches onto periodic structure but averages away sharp spikes; the AE does the exact opposite. Even a flat average of the three scores outperforms any single model, but inverse-loss weighting squeezes out another 8 points of precision on top of that.

Put it in operational terms: on a hypothetical 1-million-alert night with 0.1% real transients, cutting FPR from 0.020 to 0.009 removes about 11 000 bogus alerts from the review queue. For a ten-person team doing manual vetting, that’s the line between “manageable” and “forget it.”

The RL part solves the other headache. A locked-in 95th-percentile cutoff discards every event whose score falls below that line, and some of those are genuinely interesting dim transients. After about 30 labelled batches the PPO agent parks the threshold near 80%, which (empirically) sits closer to the actual boundary between noise and signal.

The β experiment (Table VI) shows something worth noting: inverse-loss weighting is somewhat self-correcting—even when the VAE is badly mis-configured ($\beta=1.0$), the ensemble still hits 0.847 because the weighting turns that model’s volume down. But “somewhat” is the key word; you can’t skip hyperparameter tuning entirely.

Cost-wise, three models means roughly $3\times$ the compute. On our 1 500-point data that’s 14 seconds on a laptop. For LSST volumes you’d want a GPU behind something like Triton Inference Server.

A. Threats to Validity

Three caveats. (1) Synthetic-1500 has known injection sites, so measuring precision is easy—real survey data is messier. (2) The RL agent got simulated labels, not reviews from actual astronomers. (3) We only used single-band flux; multi-band input would widen the feature space and might need different architectures.

VII. CONCLUSION

Leaning on one model is fragile; locking the threshold makes it worse. Inverse-loss weighted fusion across AE, VAE, and Transformer brought us to 0.921 precision (up from 0.811 for the best single model), and the PPO-Clip agent found a better threshold than the stock 95th percentile with under 30 labels.

We packaged the whole thing as a browser app so that an astronomer can go from raw CSV to flagged candidates without opening a terminal.

Next step: hook up a live ZTF or ANTARES feed and see how the RL agent handles distribution drift between observing seasons. We're also curious whether inverse-loss weighting holds up when the input grows from single-band flux to multi-band (g, r, i, z) photometry.

REFERENCES

- [1] E. C. Bellm *et al.*, “The Zwicky Transient Facility: System Overview, Performance, and First Results,” *PASP*, vol. 131, no. 995, p. 018002, 2019.
- [2] Ž. Ivezić *et al.*, “LSST: From Science Drivers to Reference Design and Anticipated Data Products,” *ApJ*, vol. 873, no. 2, p. 111, 2019.
- [3] M. Lochner, J. D. McEwen, H. V. Peiris, O. Lahav, and M. K. Winter, “Photometric Supernova Classification With Machine Learning,” *ApJS*, vol. 225, no. 2, p. 31, 2016.
- [4] I. Nun, K. Pichara, P. Protopapas, and D.-W. Kim, “FATS: Feature Analysis for Time Series,” *arXiv:1506.00010*, 2015.
- [5] K. L. Malanchev *et al.*, “Anomaly detection in the Zwicky Transient Facility DR3,” *MNRAS*, vol. 502, no. 4, pp. 5147–5175, 2021.
- [6] V. A. Villar *et al.*, “A deep-learning approach for live anomaly detection of extragalactic transients,” *ApJS*, vol. 255, no. 2, p. 24, 2021.
- [7] D. P. Kingma and M. Welling, “An Introduction to Variational Autoencoders,” *Foundations and Trends in ML*, vol. 12, no. 4, pp. 307–392, 2019.
- [8] S. Tuli, G. Casale, and N. R. Jennings, “TranAD: Deep Transformer Networks for Anomaly Detection in Multivariate Time Series Data,” *VLDB*, vol. 15, no. 6, pp. 1201–1214, 2022.
- [9] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv:1707.06347*, 2017.
- [10] T. T. Nguyen and V. J. Reddi, “Deep Reinforcement Learning for Cyber Security,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 9, pp. 4042–4057, 2021.
- [11] C. C. Aggarwal and S. Sathe, *Outlier Ensembles: An Introduction*. Springer, 2017.